

Как обучают большие LLM

Предварительное обучение: данные, вычисления, память и распределённый запуск

**“You have a specific budget of compute, e.g.
1024 H200s for a month.**

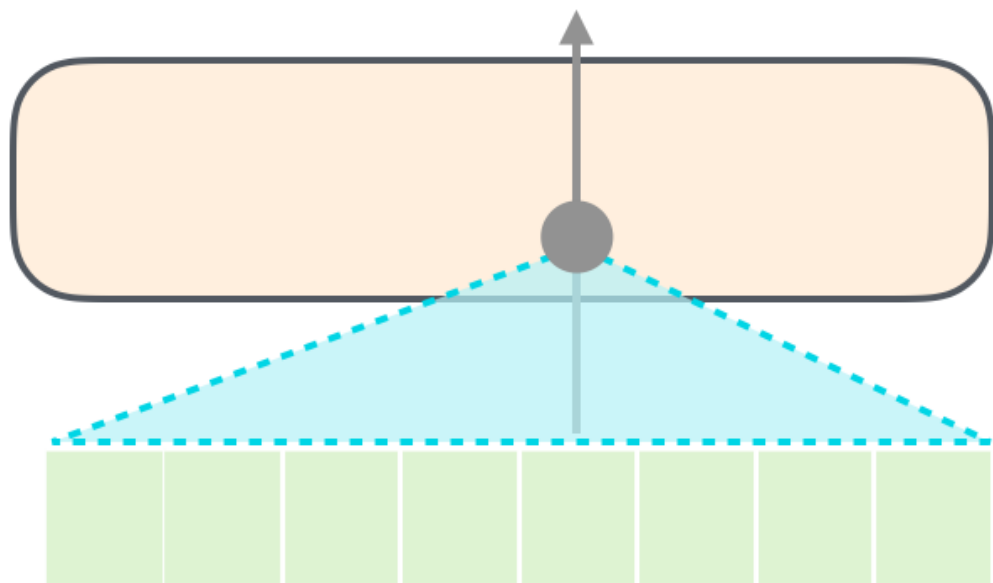
Self-Supervised Learning: What can language modeling on Web text do?

- The Eiffel Tower is located in _____, France. [factual knowledge]
- She crossed the street, checking for traffic over ____ shoulder. [syntax & coreference]
- The value I got from the two hours watching this movie was the sum total of the popcorn and the drink. The movie was _____. [sentiment]
- A shirt is \$40 but we have a 25% discount; the sale price is \$____. [math word problems]
- You are Albert Einstein. You solved this tricky physics problem... [prompting!!]
- `def solve(problem): _____` [code generation]

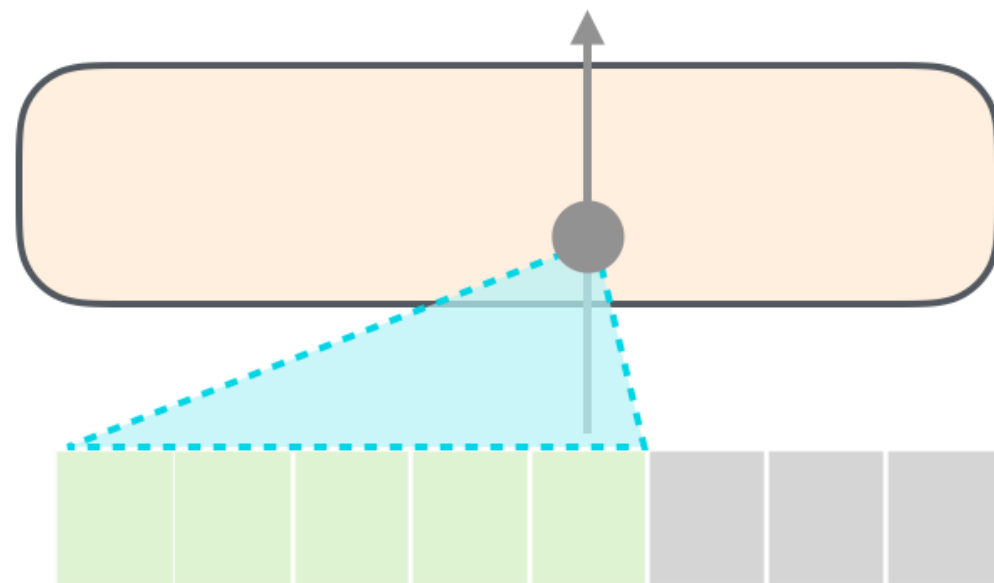
An early bet: The only way an LLM can reliably predict the next token at scale is if it can model the complexity of the phenomenon the text revolves around! (??)

Почему обучение считает позиции параллельно

Self-Attention



Masked Self-Attention



The training loop (single GPU, conceptual)

```
for step, batch in enumerate(dataloader):
    tokens = batch["input_ids"] # shape: [B, L]
    labels = batch["labels"] # shape: [B, L]

    logits = model(tokens) # [B, L, |V|]
    loss = cross_entropy(logits, labels, ignore_index=PAD)

    loss.backward() # compute gradients
    optimizer.step() # update parameters
    optimizer.zero_grad(set_to_none=True)

    if step % ckpt_every == 0:
        save_checkpoint(model, optimizer, step)
```

What changes at scale

At large scale you add: gradient accumulation, distributed collectives, mixed precision, activation checkpointing, checkpoint I/O, and evaluation.

Псевдокод: labels сдвинуты. Для PyTorch CE: $[B \cdot L, V]$ и $[B \cdot L]$.

Global batch, microbatch, and gradient accumulation

Definitions (common in large-scale pre-training)

- Microbatch: the batch processed in one forward/backward on a GPU before accumulation.
- Gradient accumulation: do A microbatches before an optimizer step.
- Data parallelism (DP): number of replicas R .

Token accounting per optimizer step

If microbatch has shape $[B, L]$:

$$B_{\text{tok}}^{\text{step}} = (R) \times (B) \times (L) \times (A).$$

Adjust for sequence packing, padding, and dropped tokens.

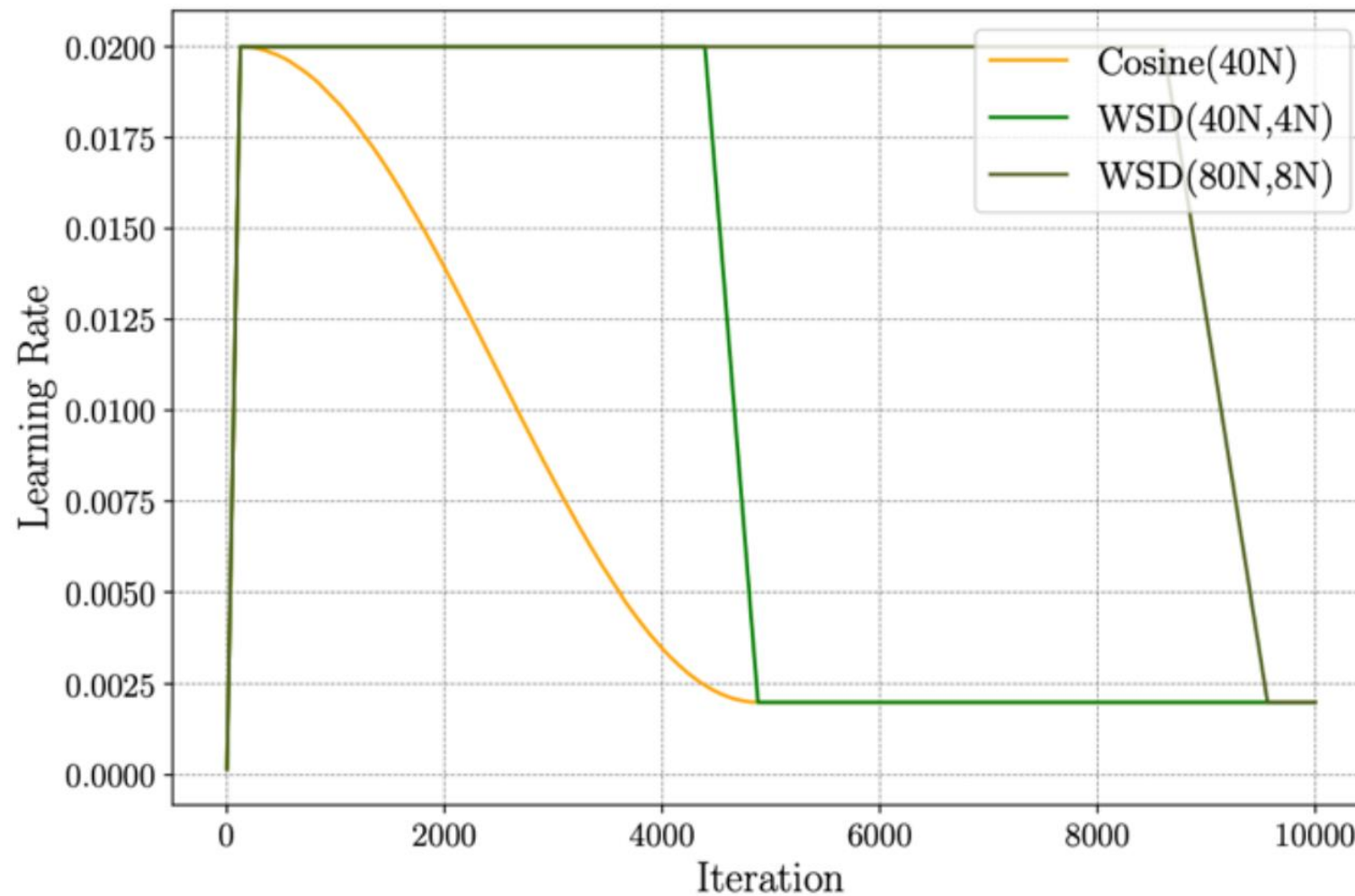
Engineering trade

Accumulation increases B_{tok} per step without increasing activation memory (if microbatch fixed), but can change optimizer dynamics (effective batch size).

“Train on the Web”

- Must **crawl pages**, starting from seed links!
- The average page is useless gibberish. How to handle this?
 - Idea #1: Over-sample certain sites! Wikipedia, arXiv, GitHub?
 - Idea #2: Filter based on known heuristics! Upvotes? PageRank?
 - Idea #3: Ask an LLM to “assess quality” and distill into a tiny model.
- Pages that remain still need a ton of cleaning and deduplication...
HTML, JavaScript, Boilerplate → clean text for next token prediction.

Learning rate: warmup, stable, decay



Compute budget couples parameters and tokens

A simple but powerful constraint

Using the dense-Transformer approximation:

$$\text{Training FLOPs} \approx 6 N_{\text{params}} N_{\text{tok}}.$$

For a fixed compute budget C :

$$6 N_{\text{params}} N_{\text{tok}} \leq C \quad \Rightarrow \quad N_{\text{tok}} \leq \frac{C}{6 N_{\text{params}}}.$$

Interpretation

If you make the model bigger (increase N_{params}) while holding compute fixed, you *must* reduce tokens. This is why pre-training is an optimization problem over:

- architecture
- data (tokens)
- compute (hardware + wall-clock)

Model FLOPs Utilization (MFU)

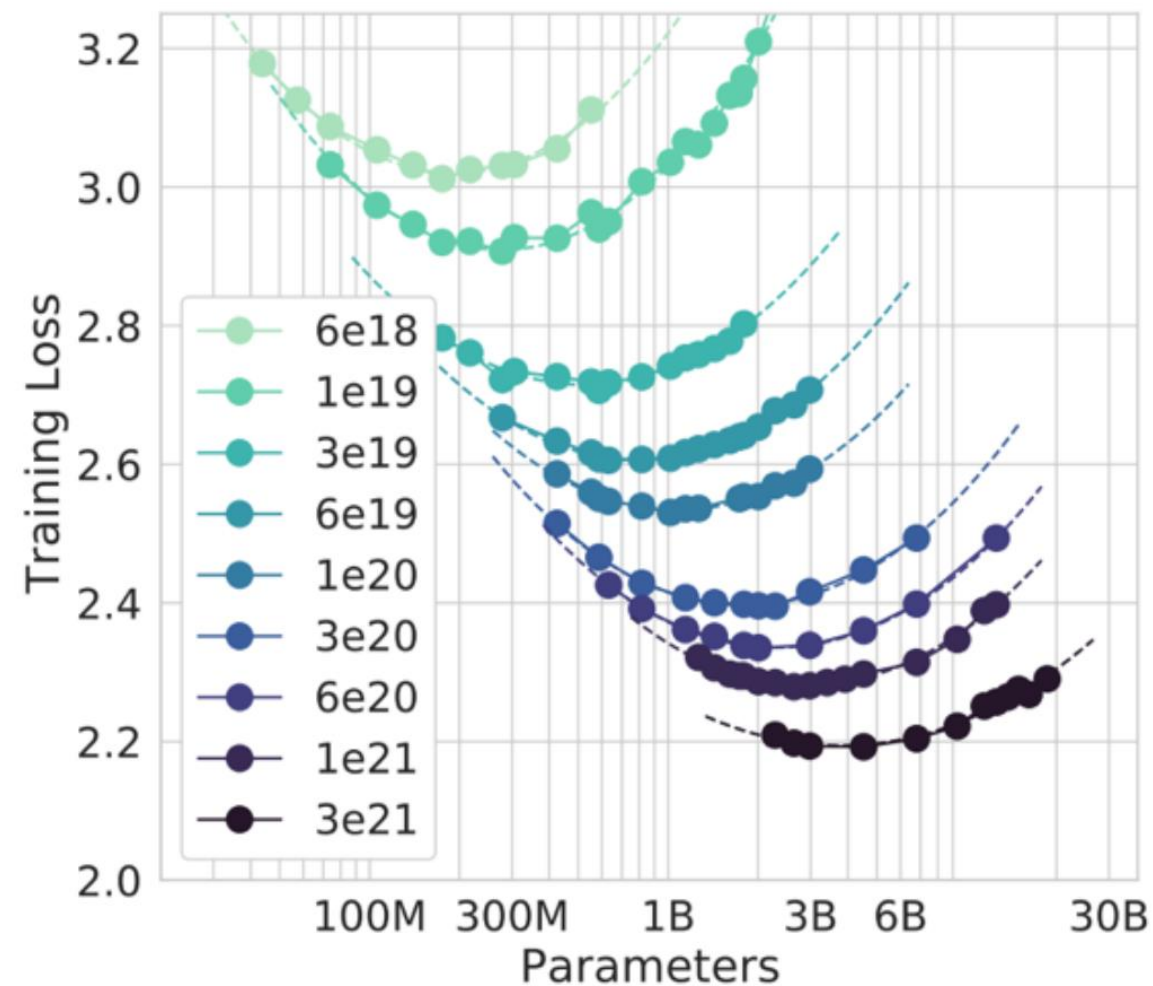
Idea: What fraction of the GPU's theoretical peak FLOPs is being used for “useful” model computation?

1. Compute $\text{FLOP}_{\text{theoretical}}$ = total number of matrix multiply FLOPs in the forward + backward pass
(can approximate backward = 2x forward)
(Ignore nonlinearities, normalization, elementwise ops like residuals. They will run on FP32 cores)
2. Look up $\text{FLOP/sec}_{\text{theoretical}}$ = theoretical max throughput of your device (H100: 989 TFLOP/sec)
3. Compute $t_{\text{theoretical}} = \text{FLOP}_{\text{theoretical}} / \text{FLOP/sec}_{\text{theoretical}}$
4. Measure t_{actual} = Actual time for a full iteration of data loading, forward, backward, optimizer step
5. $\text{MFU} = t_{\text{theoretical}} / t_{\text{actual}}$

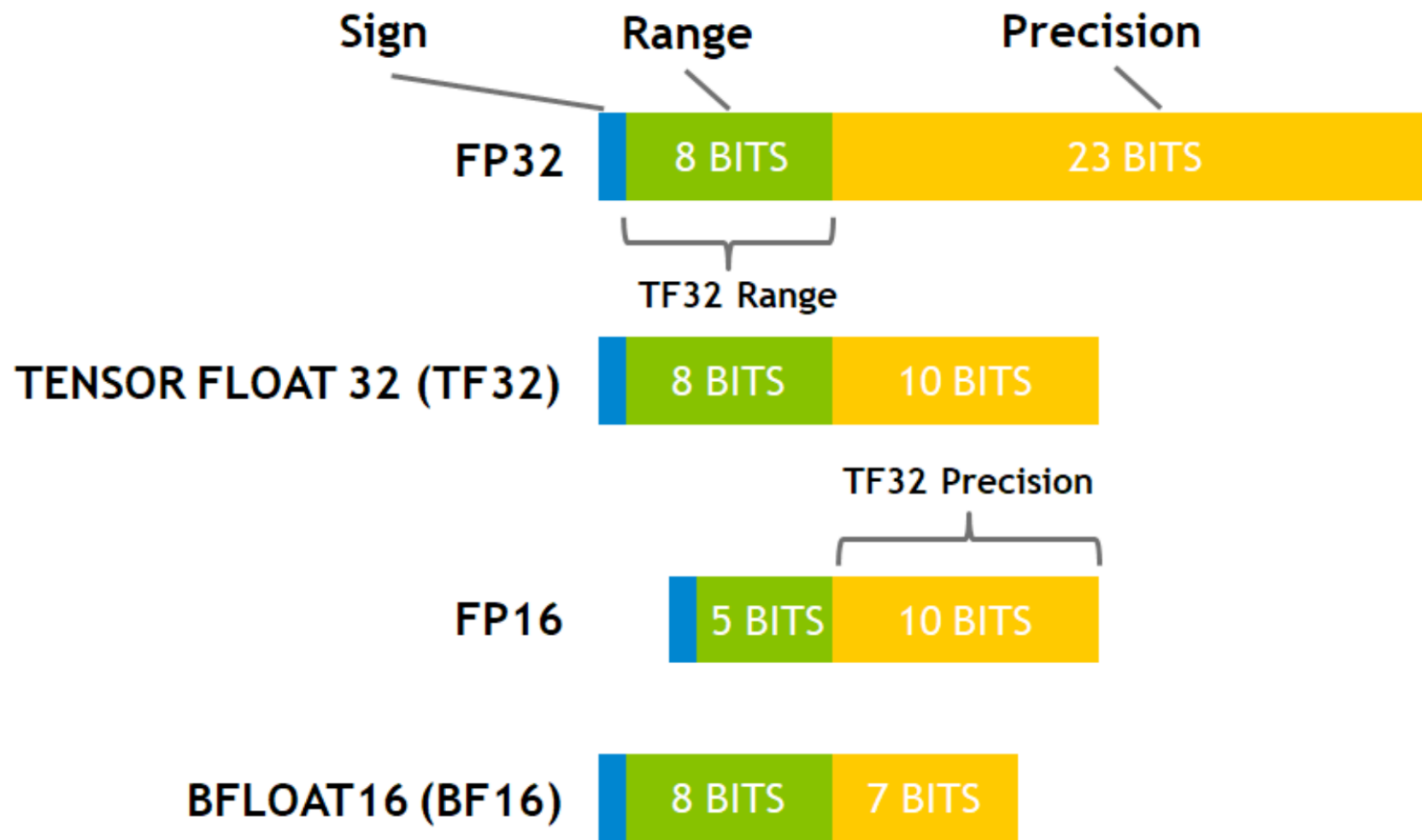
Chowdhery et al, “PaLM: Scaling Language Modeling with Pathways”, arXiv 2022

Уточнение: точность нематричных операций зависит от kernel и входов.
Для MFU сравниваем с dense peak того же численного режима.

IsoFLOP: одинаковая стоимость, разные модели



Mixed precision в обучении



Почему весов на 14 GB недостаточно для обучения 7B

FP32:

- Parameters — 4 bytes
- Gradients — 4 bytes
- Optimizer statistics — 8 bytes

16 bytes per parameter in total

AMP:

- Parameters — 2 bytes
- Master parameters — 4 bytes
- Gradients — 2 bytes (sometimes 4)
- Optimizer statistics — 8 bytes

Also **16 bytes per parameter!**

Activation Checkpointing

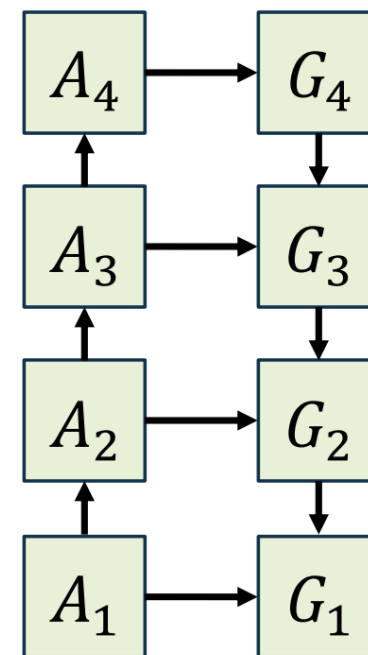
Each layer in the network is two functions:

Forward: Compute next-layer activations

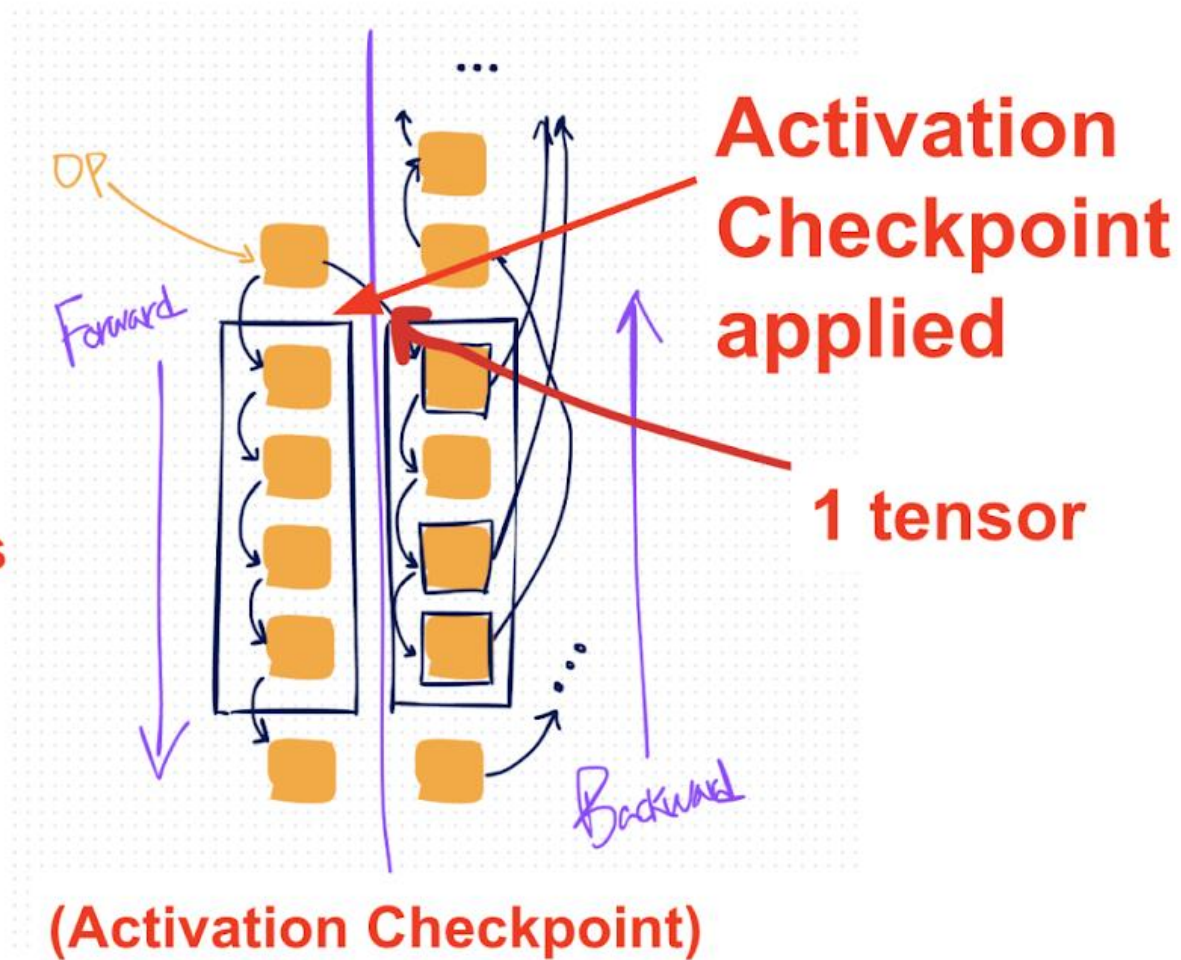
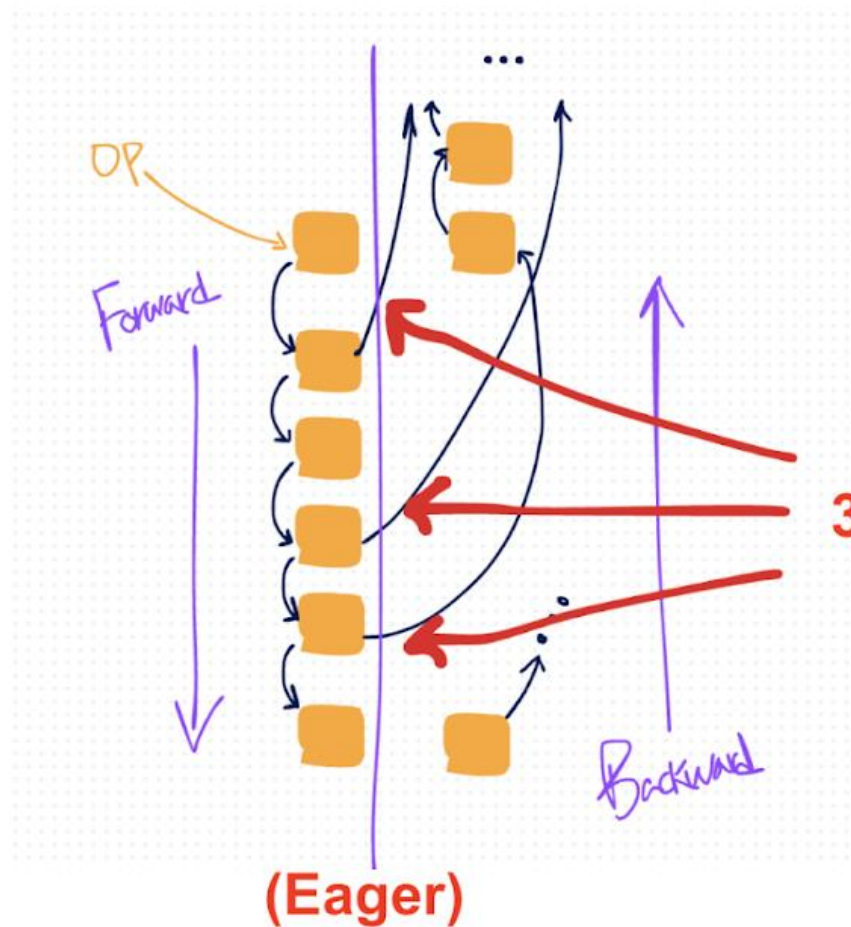
$$A_{i+1} = F_i^{\rightarrow}(A_i)$$

Backward: Compute prev-layer gradients

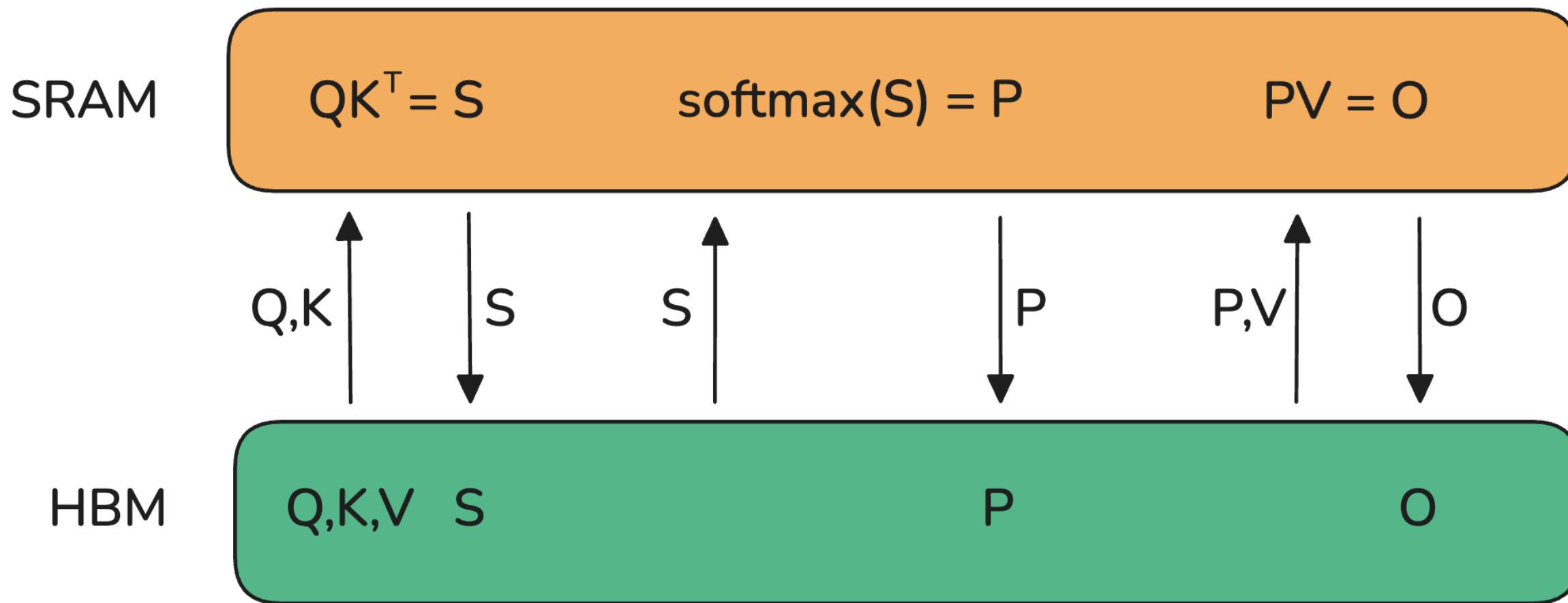
$$G_i = F_i^{\leftarrow}(A_i, G_{i+1})$$



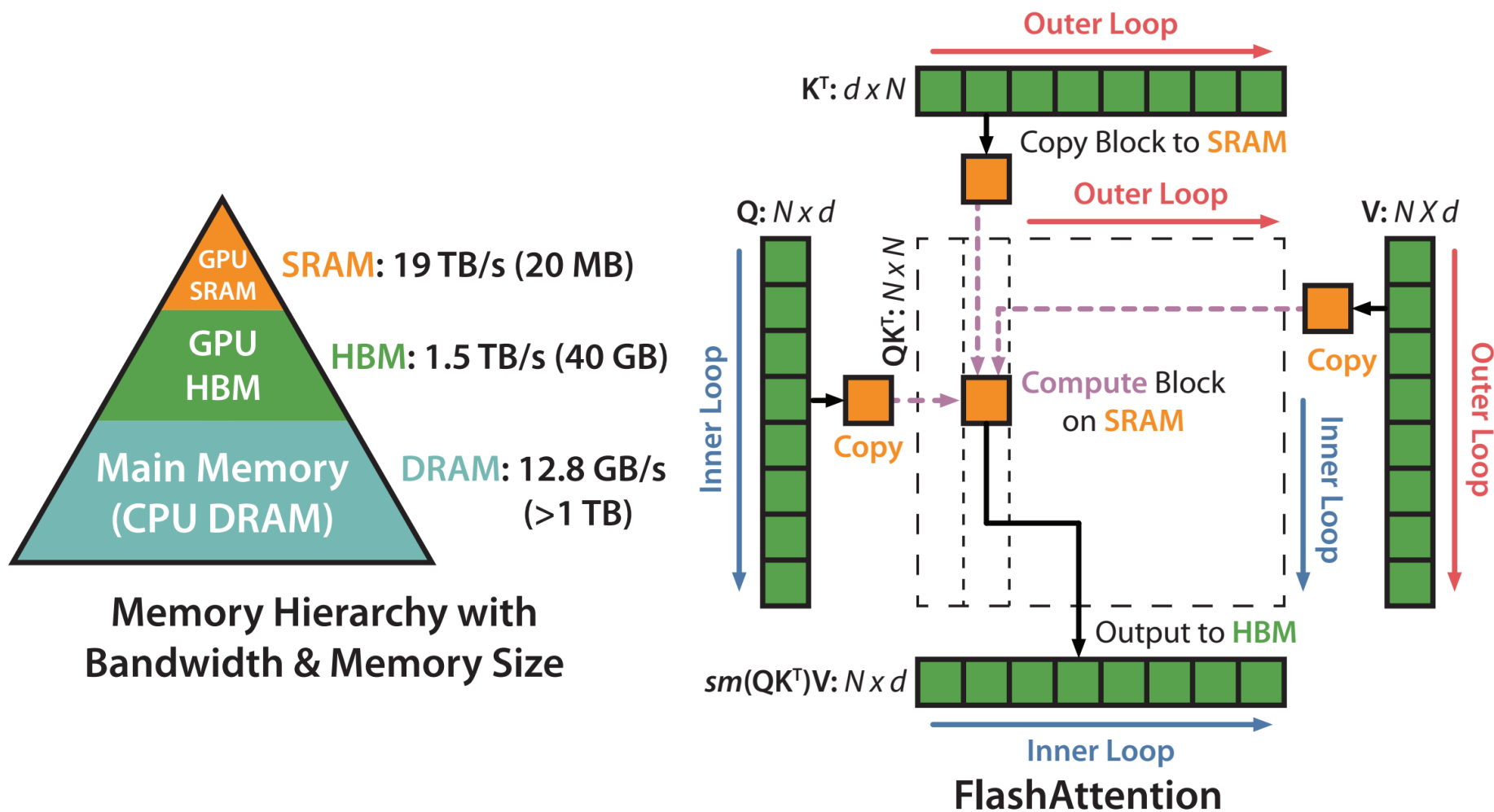
Activation checkpointing



Attention: цена промежуточных матриц



FlashAttention: точный attention блоками



Case Study: Meta's Llama3 Cluster

H100 GPU

3352 GB/sec inside the GPU

Server = 8x GPU

900 GB/sec between GPUs

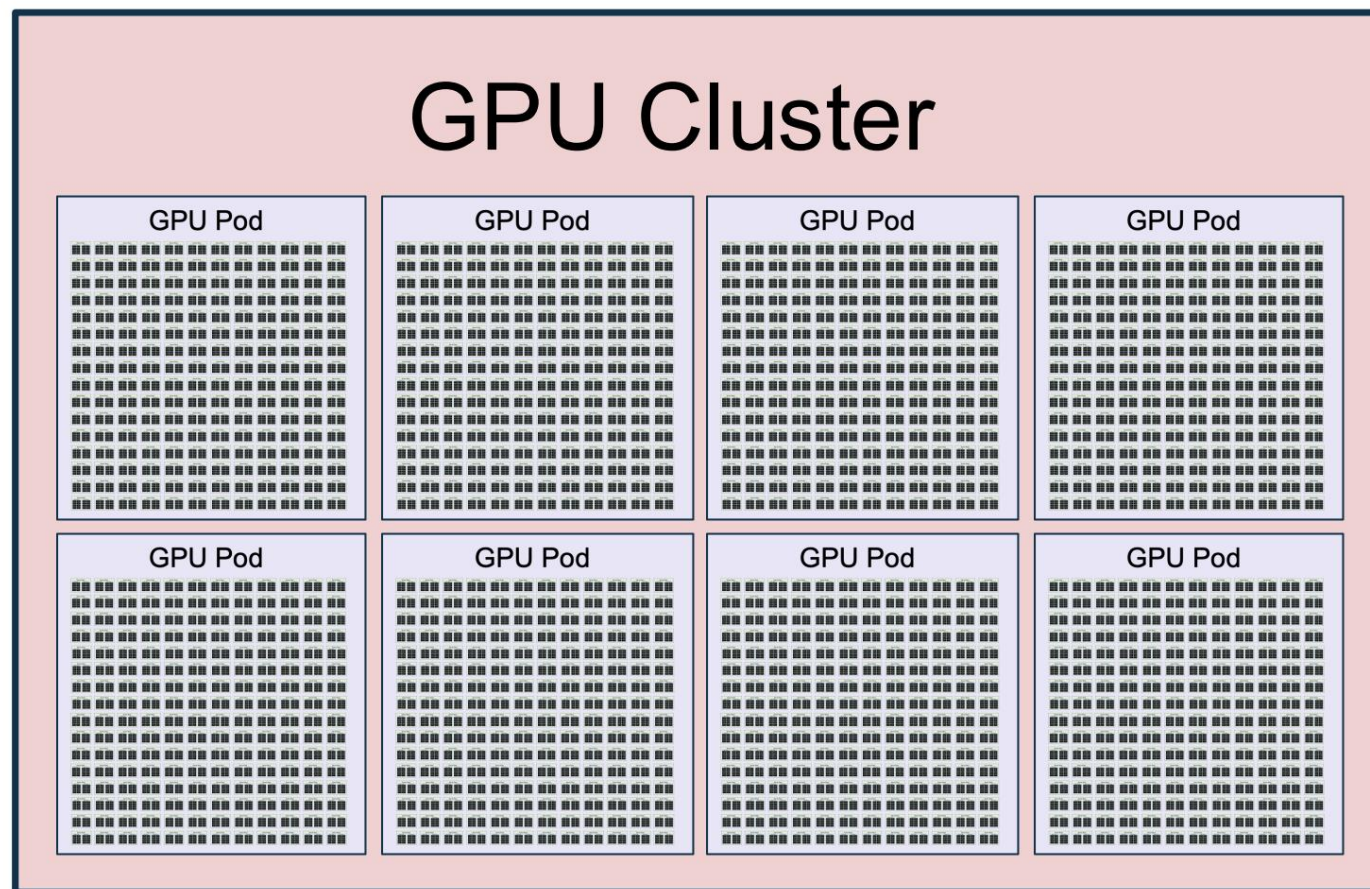
Rack = 2 Servers = 16x GPU

Pod = 192 Racks = 3072 GPUs

50 GB/sec between GPUs

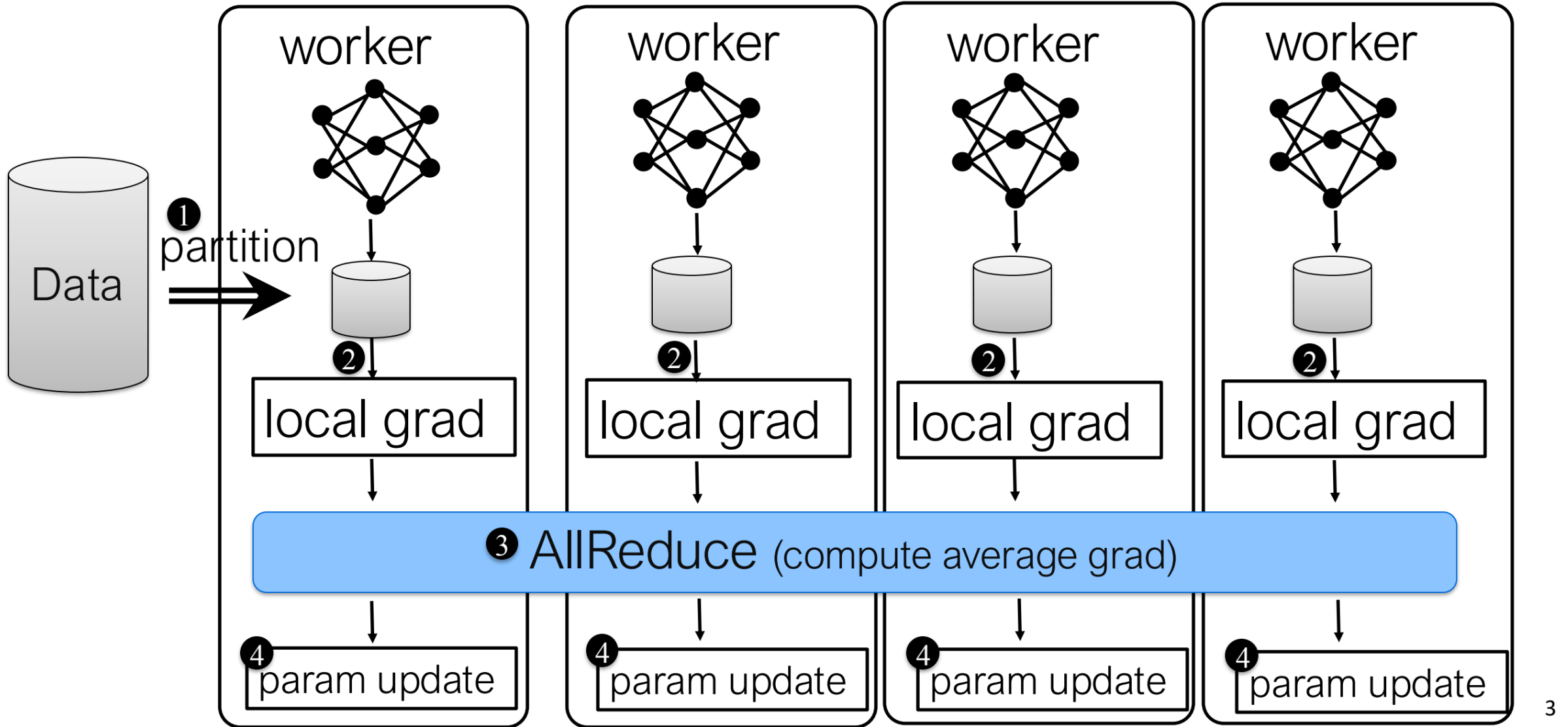
Cluster = 8 Pods = 24,576 GPUs

< 50GB/sec between GPUs



Llama Team, "The Llama 3 Herd of Models", <https://arxiv.org/abs/2407.21783>

Distributed Data Parallel Training



Visualizing REDUCESCATTER

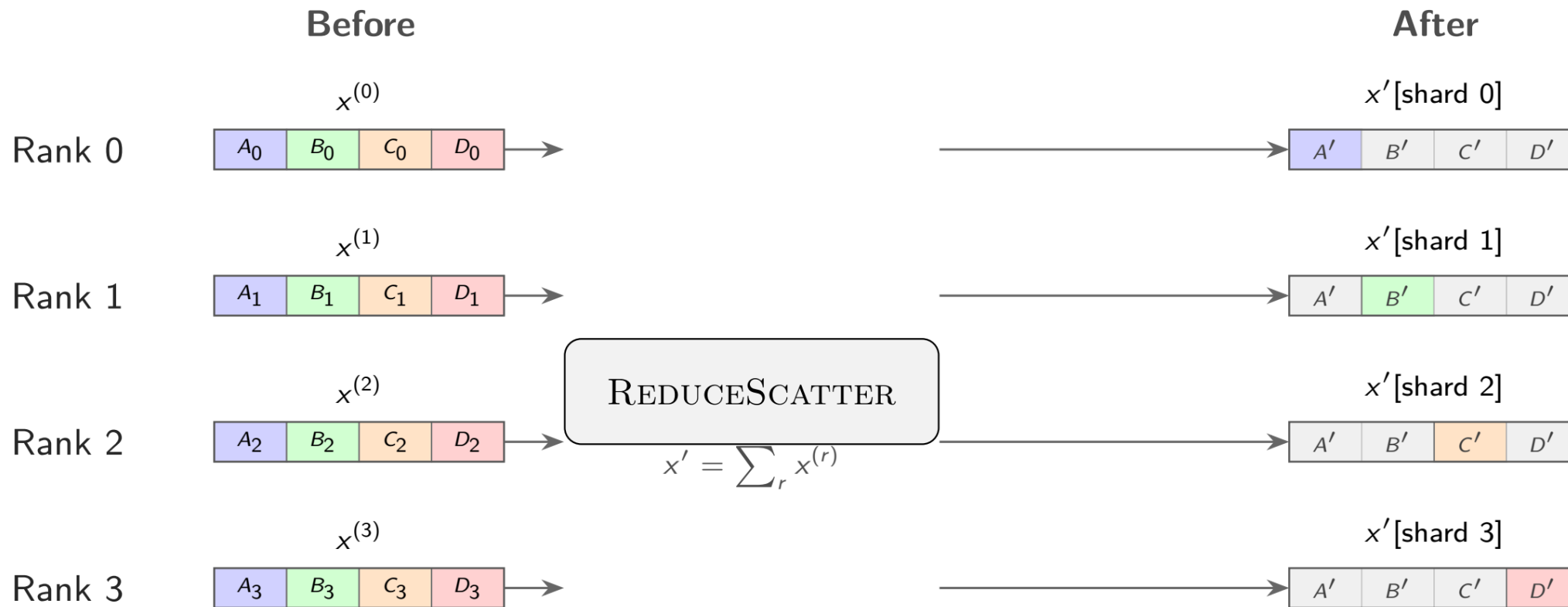


Figure: Same reduction as ALLREDUCE, but each rank keeps only its shard (e.g. rank 1 keeps B').

Visualizing ALLGATHER

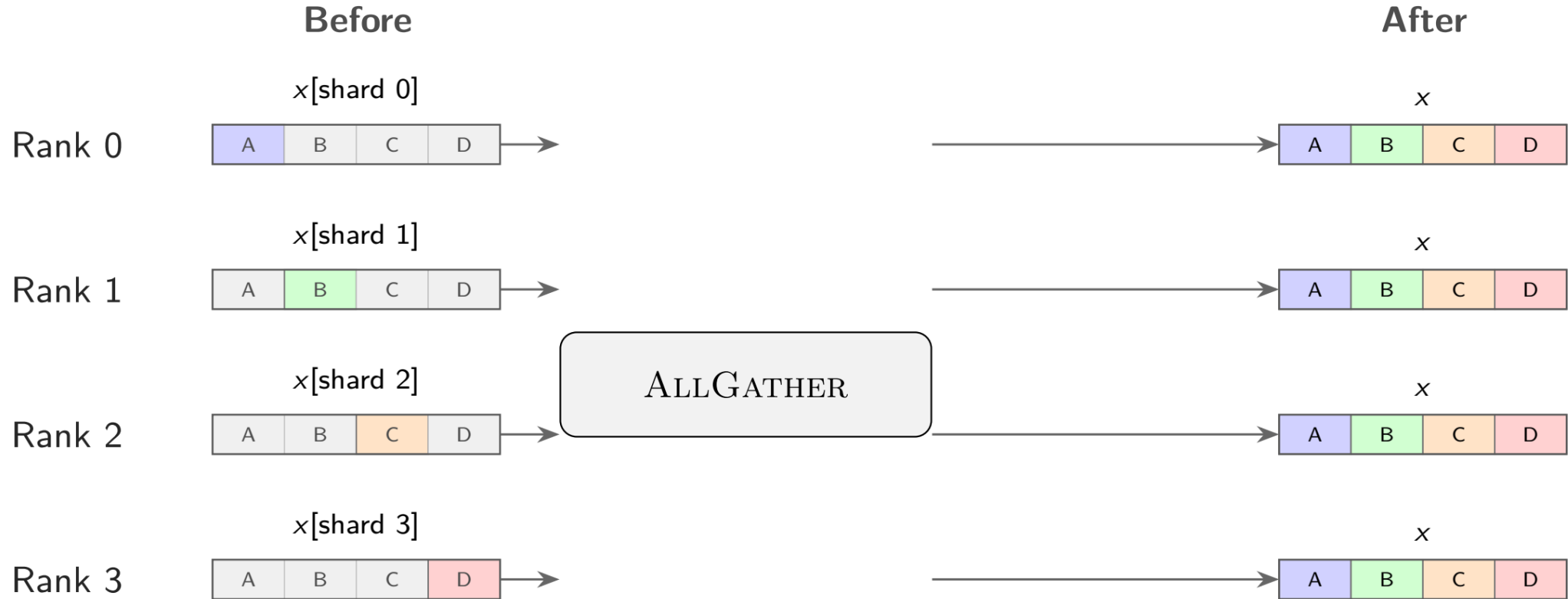
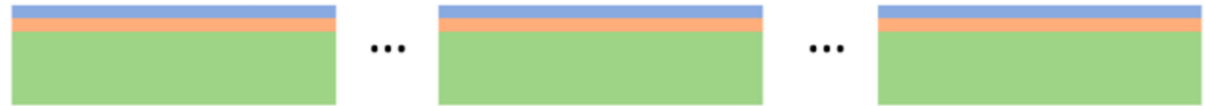
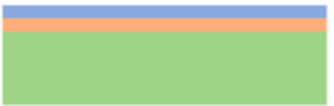
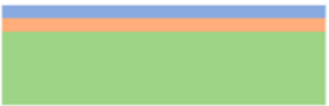
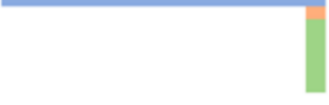


Figure: Each rank starts with a shard; the group gathers so every rank reconstructs the full tensor.

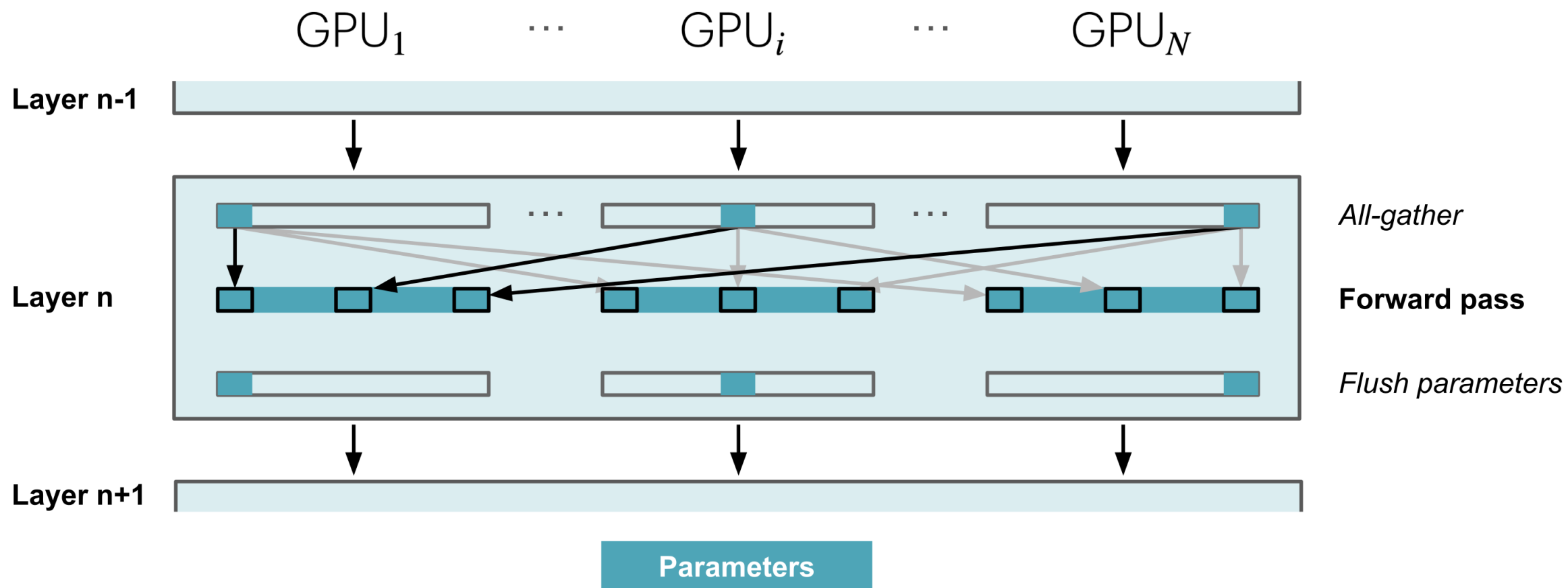
ZeRO – solving the memory overhead issue of DP

Core idea: split up the expensive parts (state) and use the reduce-scatter equivalence.

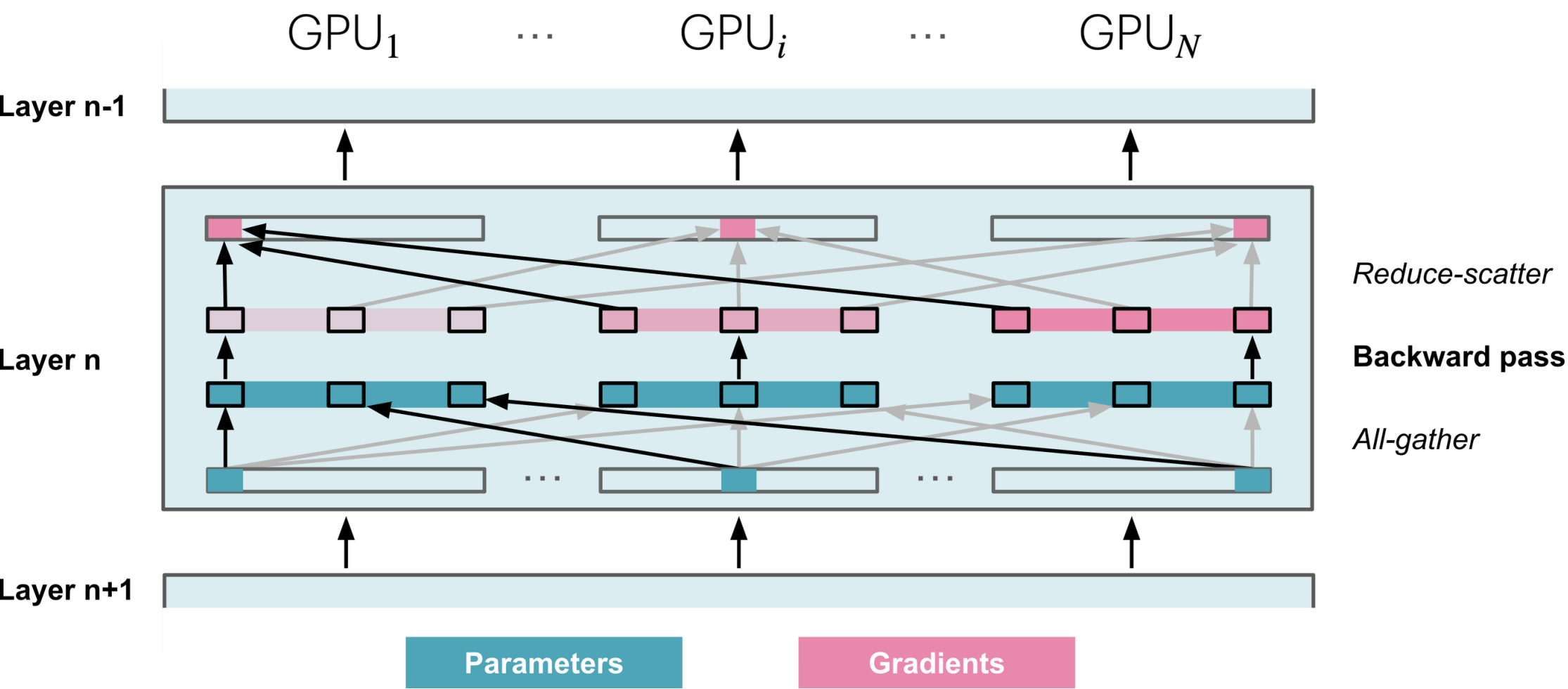
	gpu ₀	...	gpu _i	...	gpu _{N-1}	Memory Consumed	K=12 Ψ=7.5B N _d =64
Baseline		...		...		$(2 + 2 + K) * \Psi$	120GB
P _{os}		...		...		$2\Psi + 2\Psi + \frac{K * \Psi}{N_d}$	31.4GB
P _{os+g}		...		...		$2\Psi + \frac{(2 + K) * \Psi}{N_d}$	16.6GB
P _{os+g+p}		...		...		$\frac{(2 + 2 + K) * \Psi}{N_d}$	1.9GB

 Parameters
 Gradients
 Optimizer States

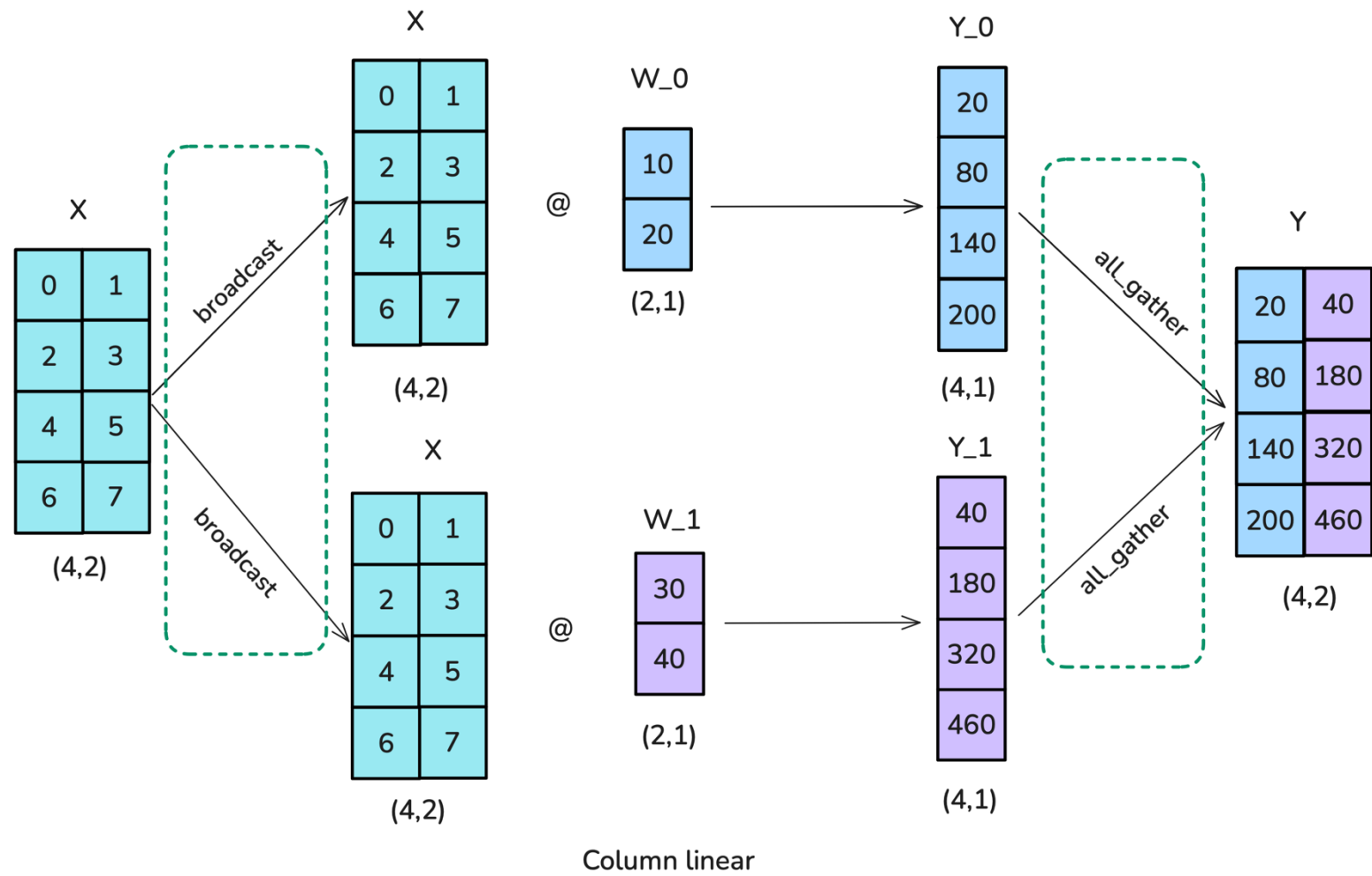
FSDP: форвард пасс



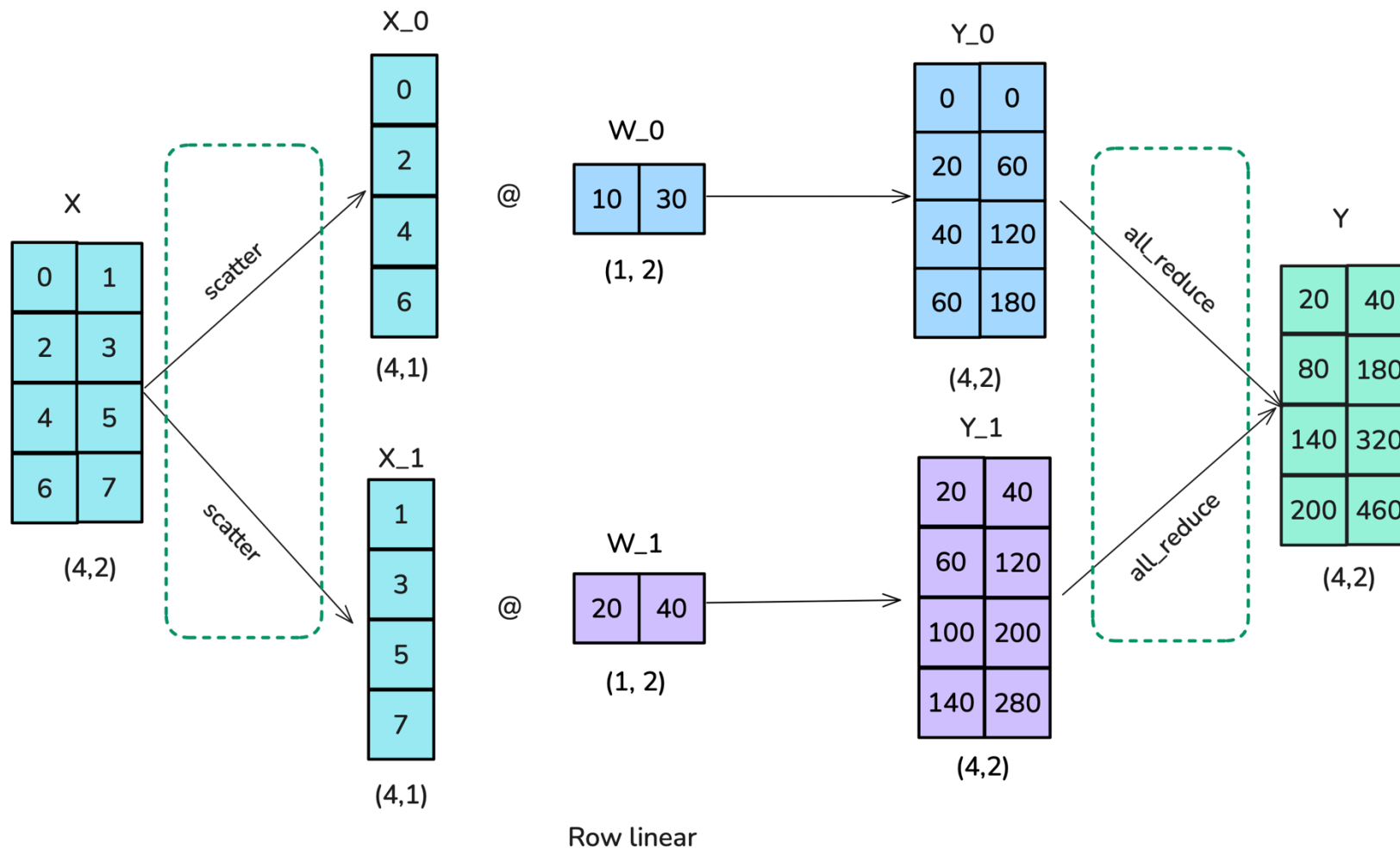
FSDP: бэкпроп и обновление



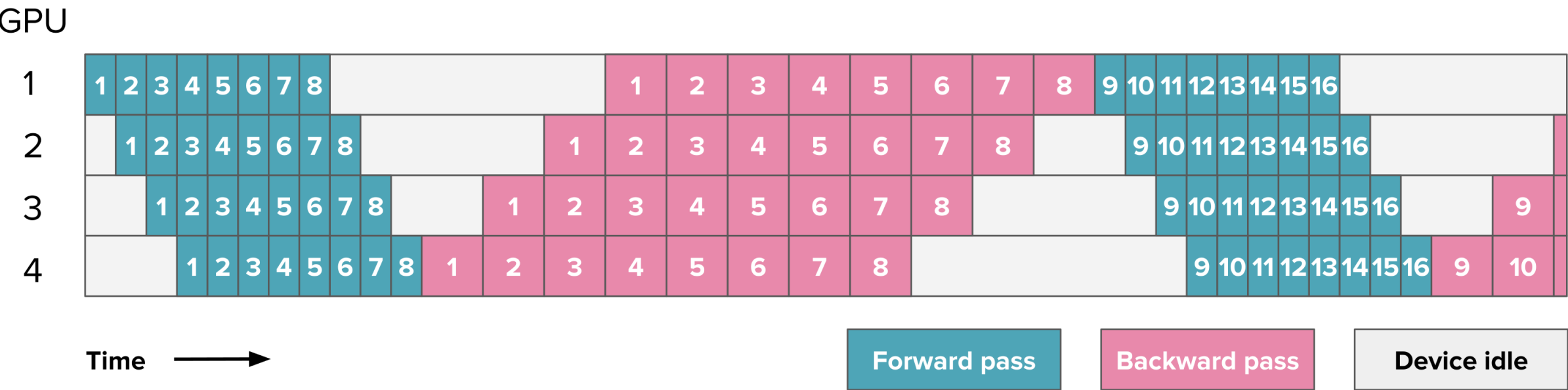
TP: деление матрицы по столбцам



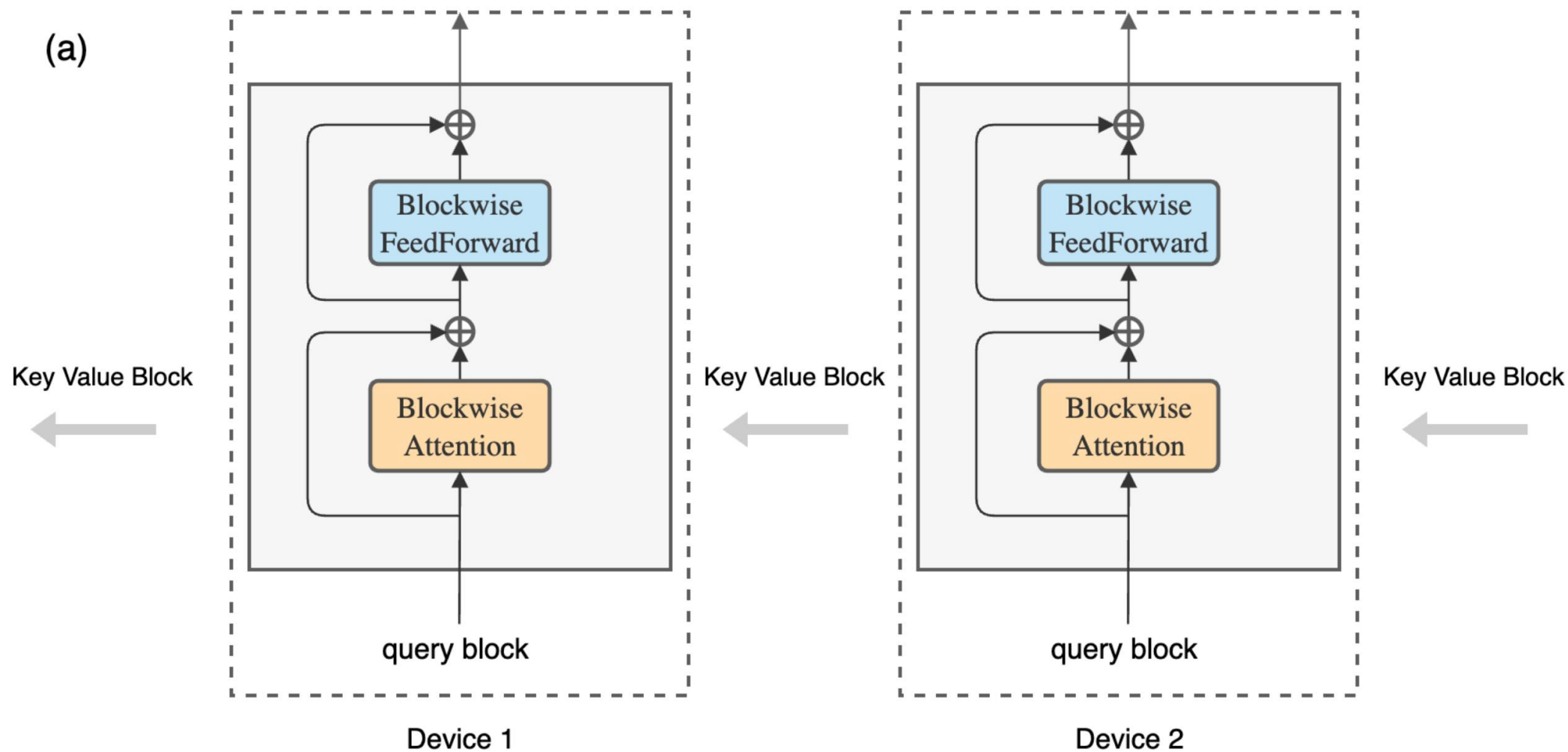
TP: деление матрицы по строкам



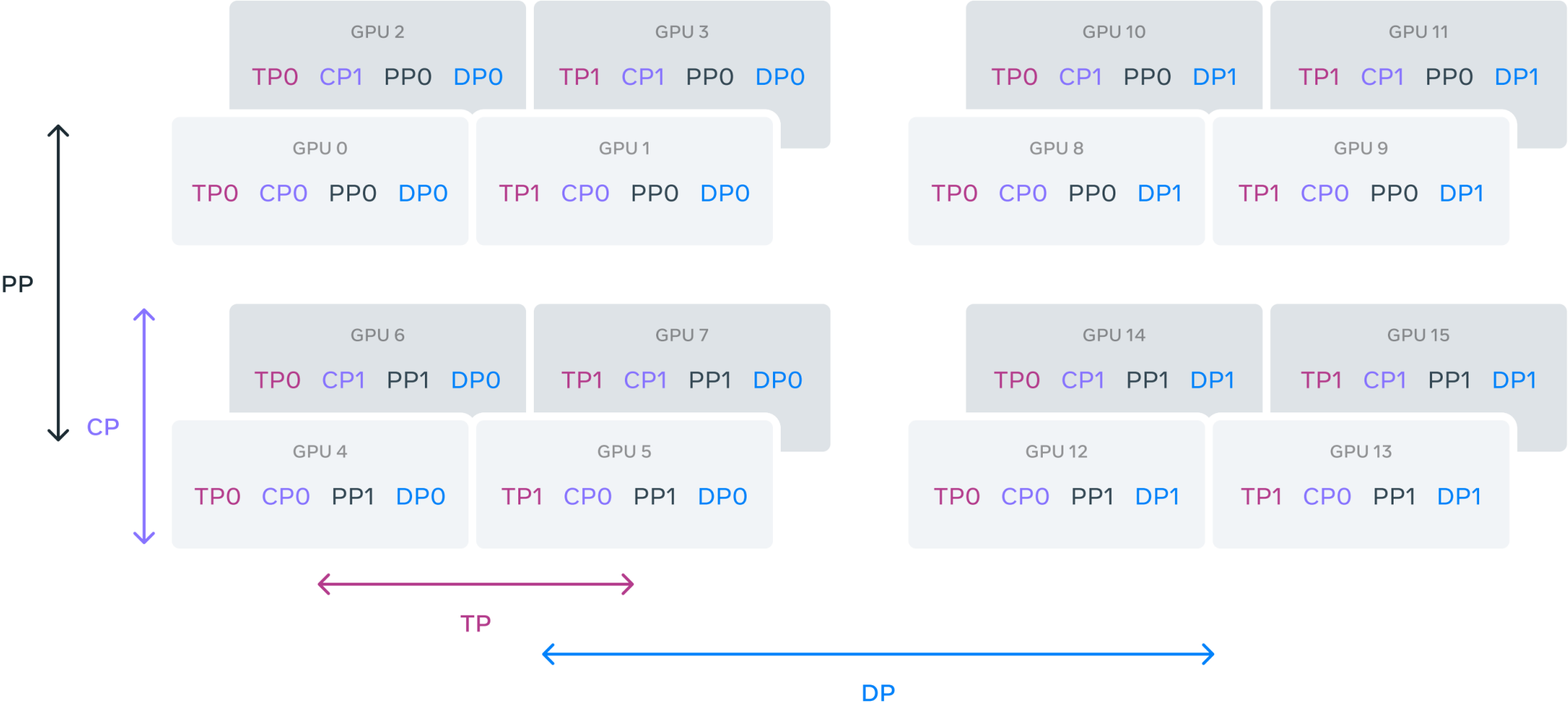
Pipeline parallelism и микробатчи



Context parallelism



Как сочетаются DP, TP, PP и CP



Llama 3 405B: 8K и 128K контекст

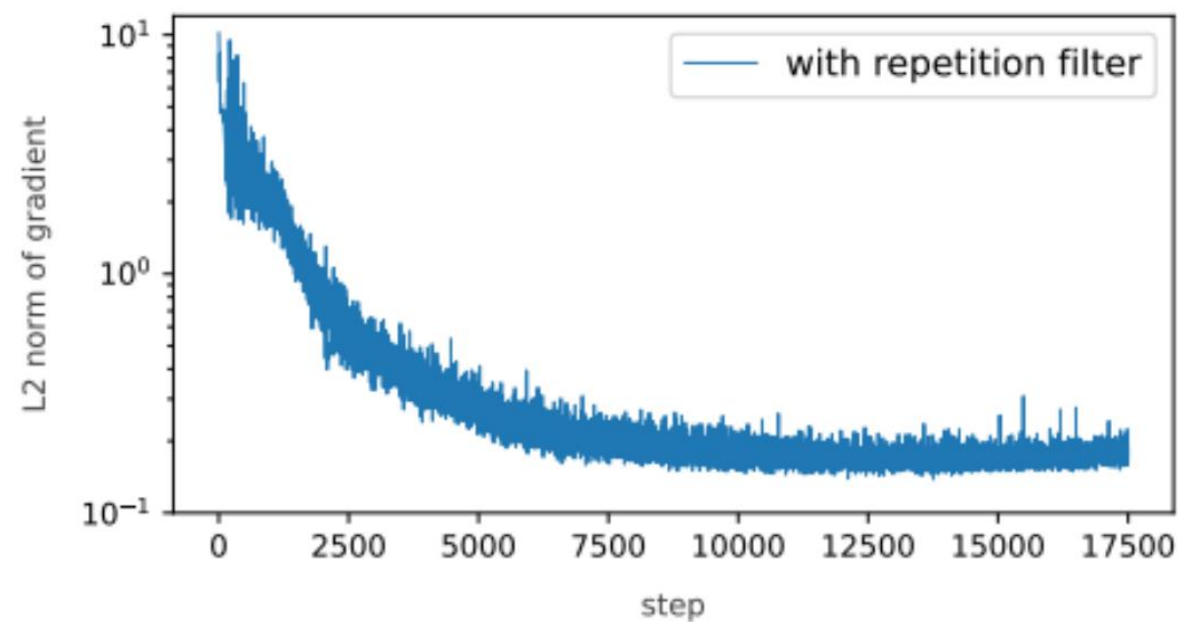
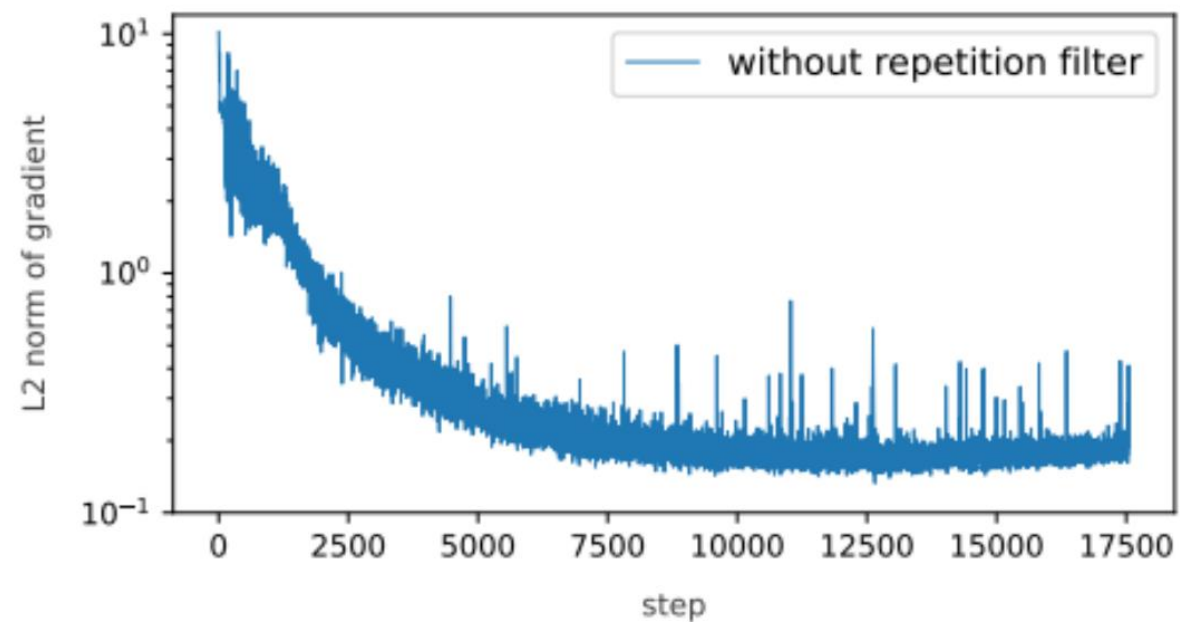
GPUs	TP	CP	PP	DP	Seq. Len.	Batch size/DP	Tokens/Batch	TFLOPs/GPU	BF16 MFU
8,192	8	1	16	64	8,192	32	16M	430	43%
16,384	8	1	16	128	8,192	16	16M	400	41%
16,384	8	16	16	8	131,072	16	16M	380	38%

Table 4 Scaling configurations and MFU for each stage of Llama 3 405B pre-training. See text and Figure 5 for descriptions of each type of parallelism.

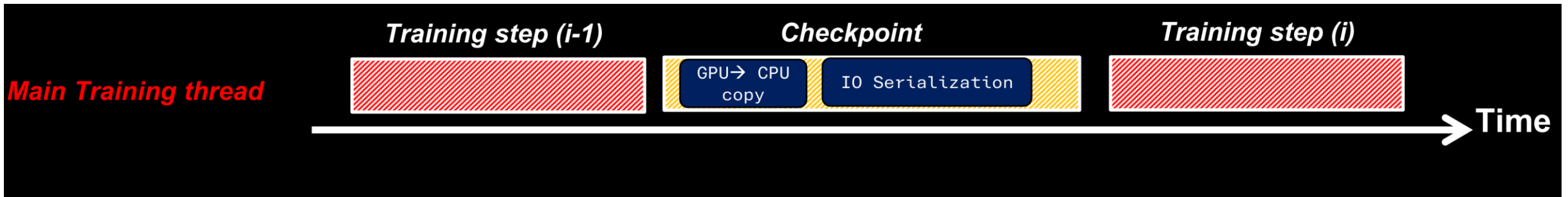
Профиль шага



Скачки градиента

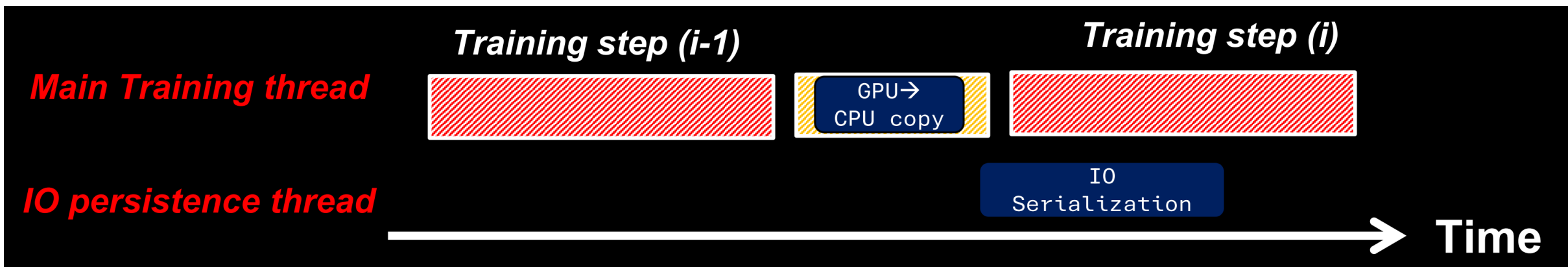


Синхронное сохранение checkpoint



Следующий шаг ждёт копирования и завершения записи.

Асинхронное сохранение checkpoint



Запись продолжается параллельно обучению.
При сбое доступен только завершённый checkpoint.

Сколько токенов успеем за месяц

Effective tokens/day (per GPU)

$$\text{tokens/day/GPU} \approx \underbrace{\text{goodput}}_{\text{time fraction}} \cdot \underbrace{\frac{\text{MFU } C_{\text{peak}}}{F_{\text{tok}}}}_{\text{tokens/sec while computing}} \cdot 86400.$$

Cluster-level (for G GPUs)

$$\text{tokens/day (cluster)} \approx G \cdot \text{goodput} \cdot \frac{\text{MFU } C_{\text{peak}}}{F_{\text{tok}}} \cdot 86400.$$

Учебные допущения: 70В, весь кластер 0,8 млн токенов/с, рабочее время 90%
62,208 млрд токенов/сутки × 30 суток = 1,866Т токенов

Срок учебного запуска 70B

Pre-training planning equation

Let $N_{\text{tok}}^{\text{target}}$ be your target training tokens.

$$\text{days} \approx \frac{N_{\text{tok}}^{\text{target}}}{\text{tokens/day (cluster)}}.$$

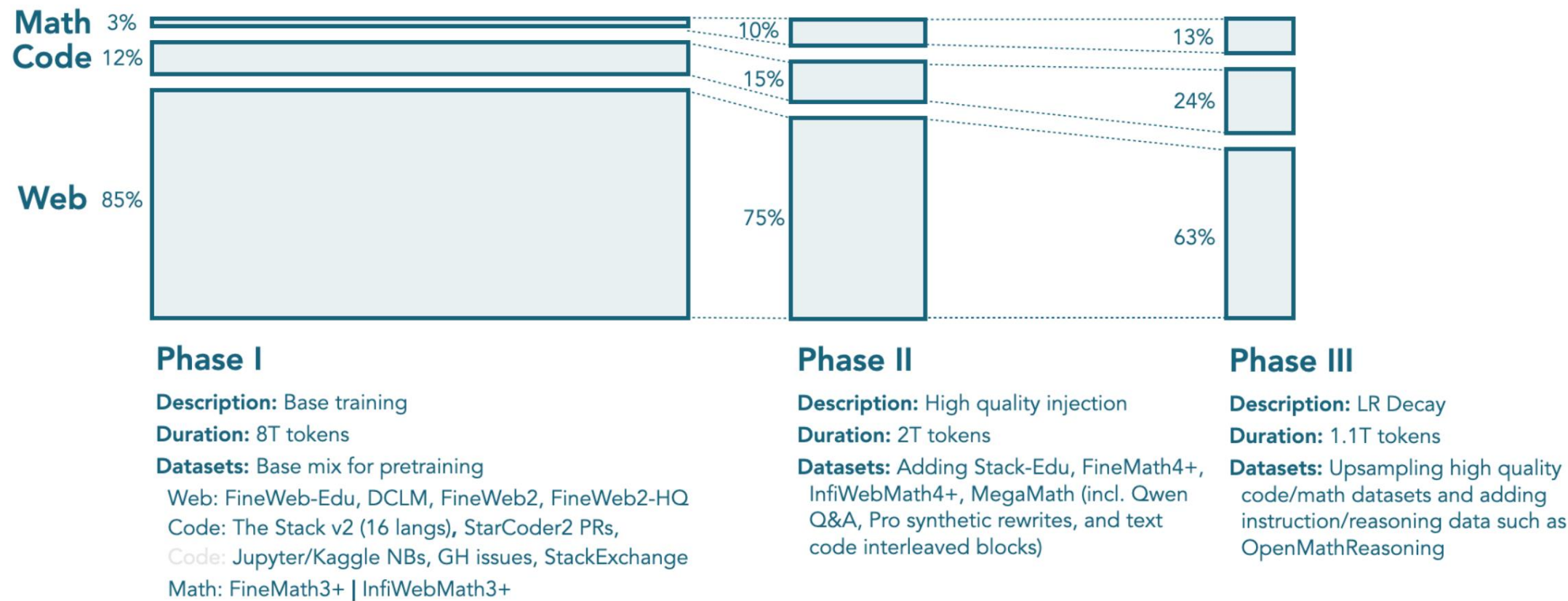
Условный расчёт: 1,5Т токенов за 24,1 дня

При 0,6 млн токенов/с: 32,2 дня

Срок оценён. Скорость и качество ещё проверяем.

Example of Data
Details. For SmolLM3.

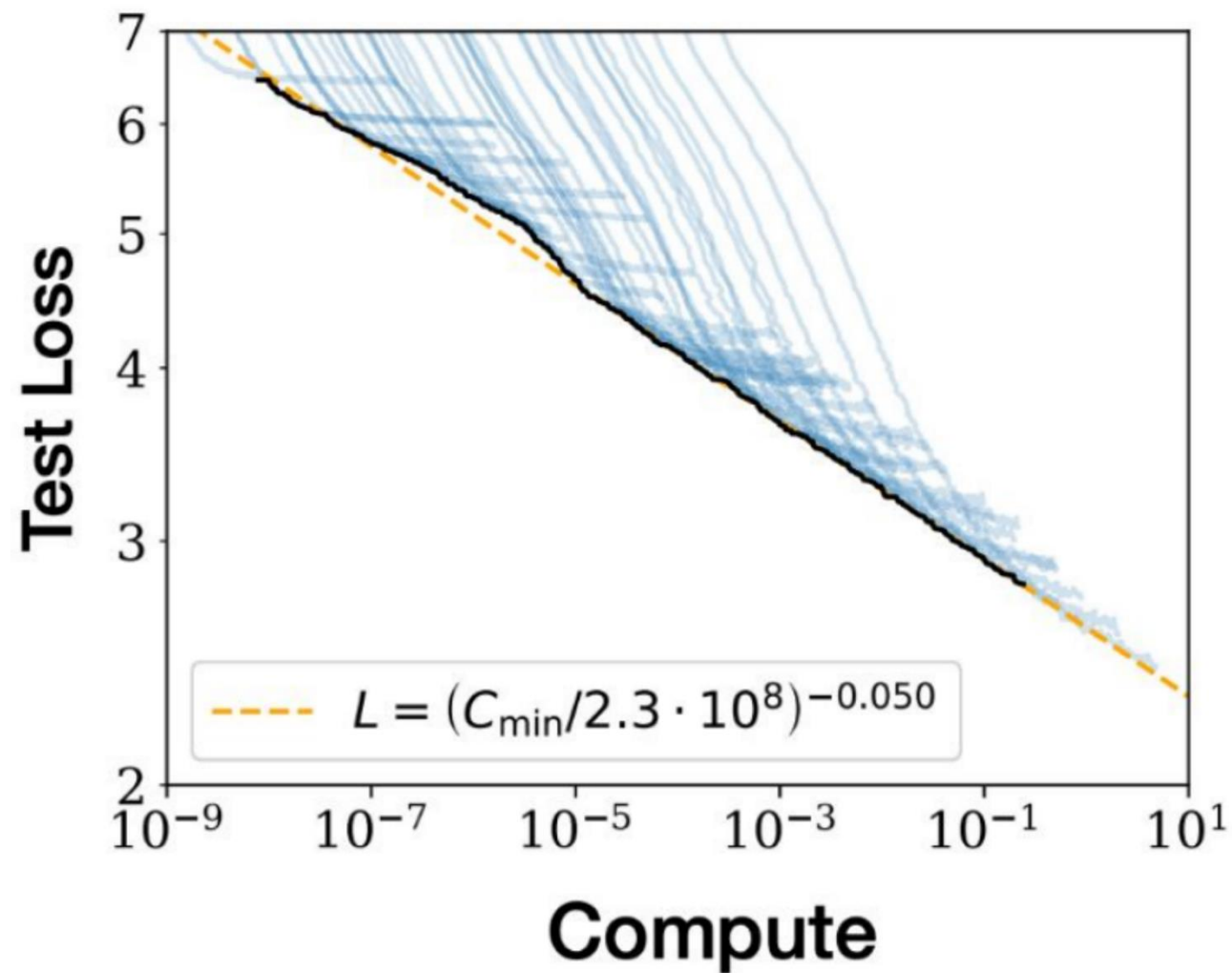
Pretraining Recipe



LR Schedule



Что позволяют предсказать scaling laws



How does the loss scale with **parameters (N)** and **data (D)**?

$$L(N, D) = E + \frac{A}{N^\alpha} + \frac{B}{D^\beta}$$

L : Negative log-likelihood (log perplexity)

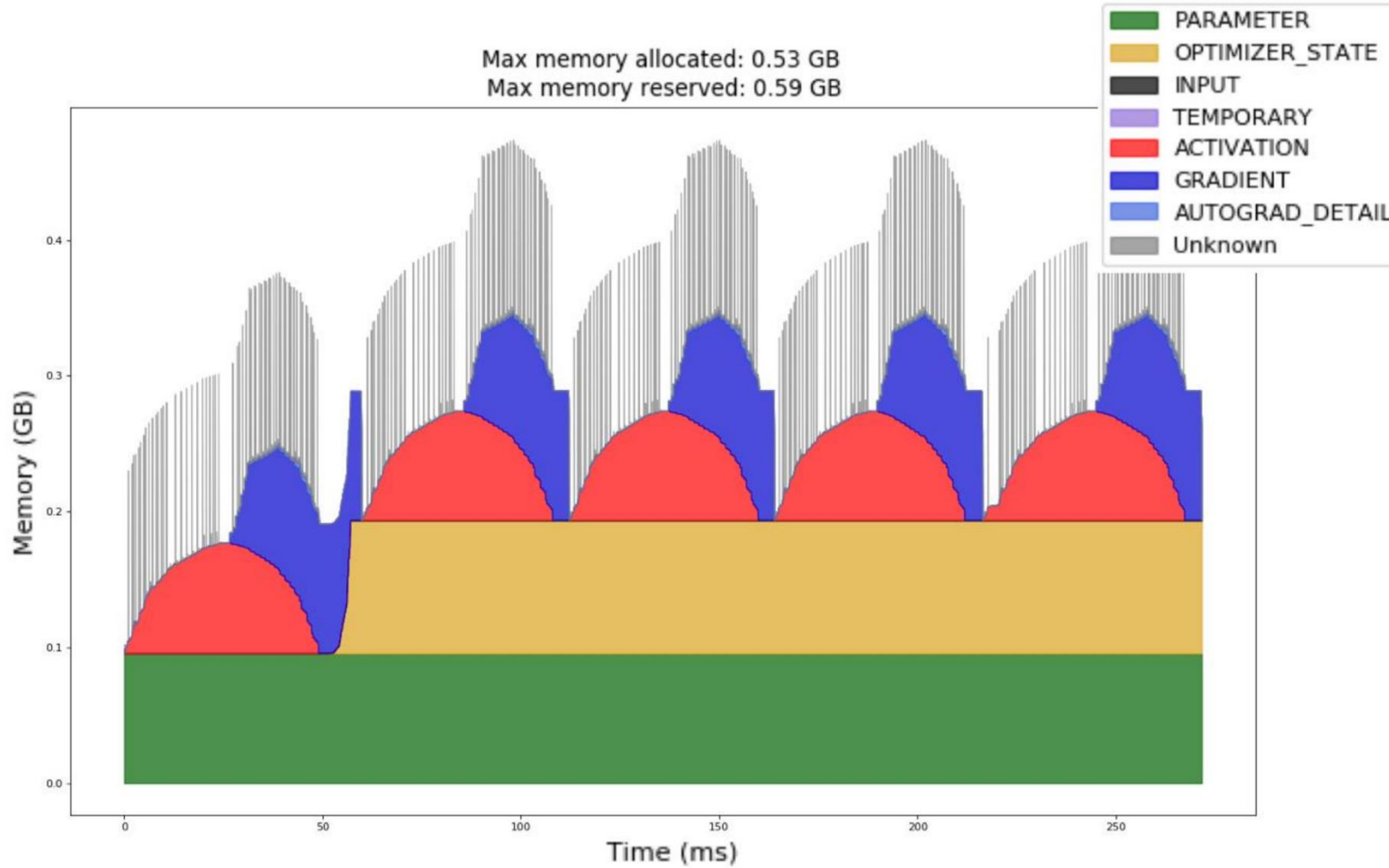
N : Number of parameters

D : Number of tokens

E, A, B, α, β : Fitted values

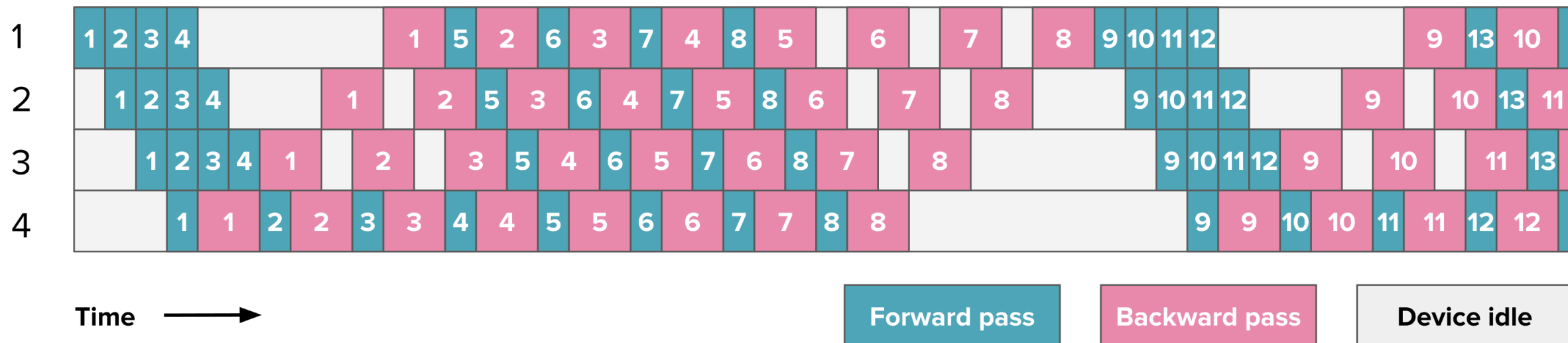
“Chinchilla” Training Compute-Optimal Large Language Models (2022)

Память меняется во время шага

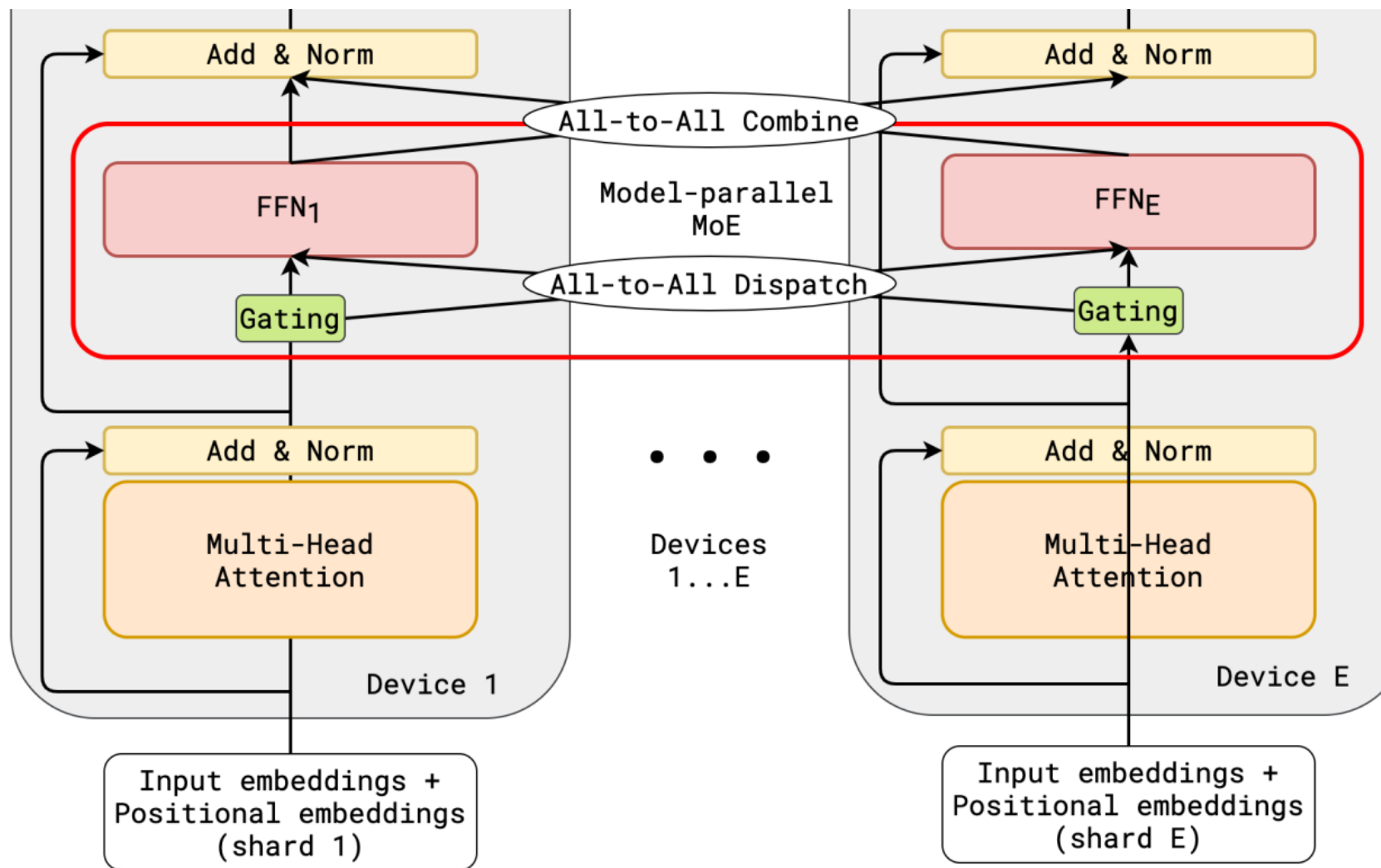


Расписание 1F1B

GPU



Expert parallelism: dispatch и combine



Отказы во время обучения Llama 3

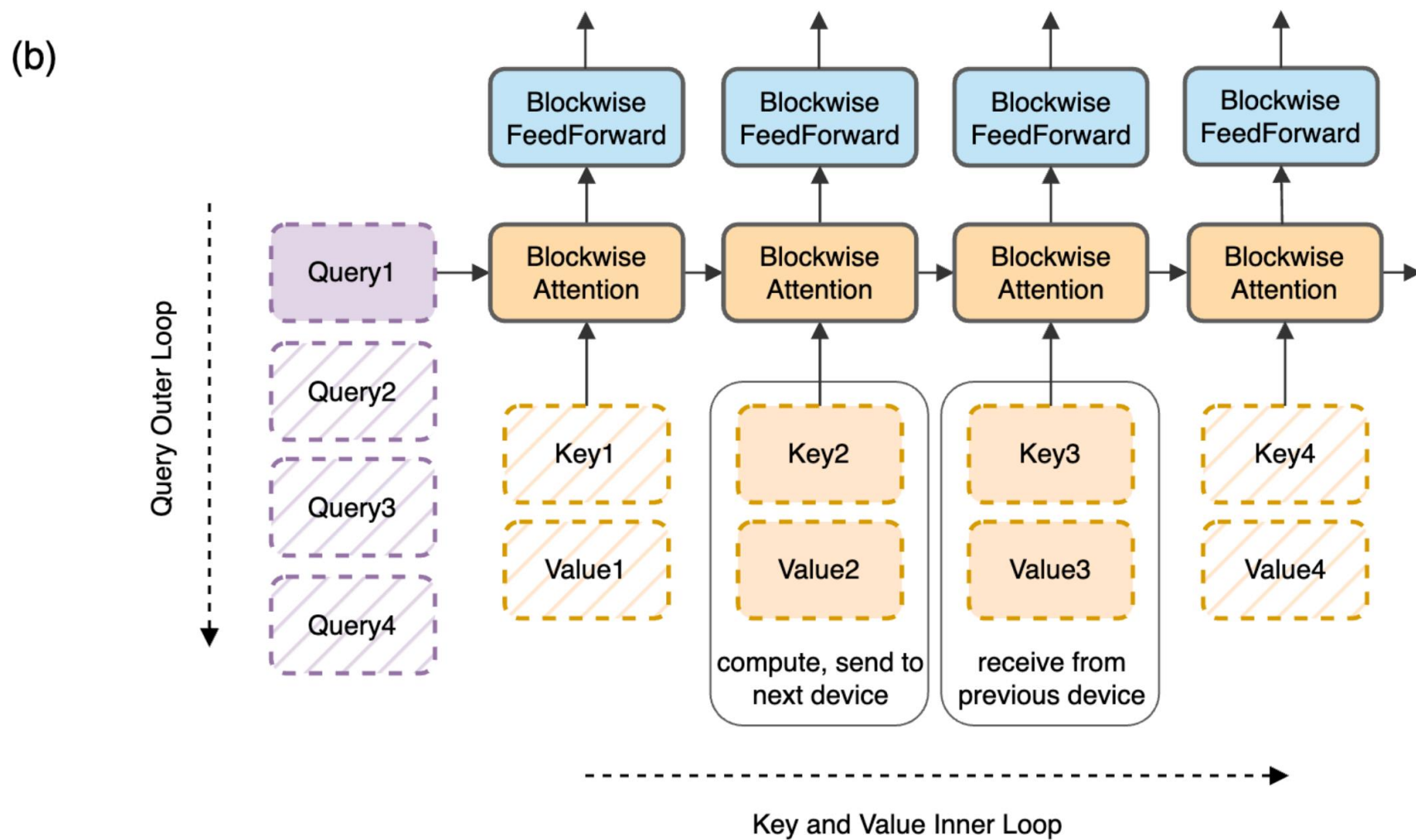
Component	Category	Interruption Count	% of Interruptions
Faulty GPU	GPU	148	30.1%
GPU HBM3 Memory	GPU	72	17.2%
Software Bug	Dependency	54	12.9%
Network Switch/Cable	Network	35	8.4%
Host Maintenance	Unplanned Maintenance	32	7.6%
GPU SRAM Memory	GPU	19	4.5%
GPU System Processor	GPU	17	4.1%
NIC	Host	7	1.7%
NCCL Watchdog Timeouts	Unknown	7	1.7%
Silent Data Corruption	GPU	6	1.4%
GPU Thermal Interface + Sensor	GPU	6	1.4%
SSD	Host	3	0.7%
Power Supply	Host	3	0.7%
Server Chassis	Host	2	0.5%
IO Expansion Board	Host	2	0.5%
Dependency	Dependency	2	0.5%
CPU	Host	2	0.5%
System Memory	Host	2	0.5%

Table 5 Root-cause categorization of unexpected interruptions during a 54-day period of Llama 3 405B pre-training. About 78% of unexpected interruptions were attributed to confirmed or suspected hardware issues.

Опечатка оригинала: Faulty GPU — 148 / 419 ≈ 35,3%, а не напечатанные 30,1%.

Dubey et al., The Llama 3 Herd of Models (2024)

Дополнение: Ring Attention



UCP: Accuracy

Recovery from checkpoint needs to be accurate, fast and agnostic to changing parallelism patterns

Blue denotes the actual training run loss, and orange denotes the loss after checkpoint recovery with changing parallelism

Experiments over GPU clusters and remote high-performance NVMe storage system.



Initial Configuration (blue) : 8 MI210 nodes (32 GPUs) with TP=4, PP=4, DP=2.
Resume Training (orange) : 4 MI210 nodes (16 GPUs) with TP=2, PP=8, DP=1.

UCP enables failure recovery with resource rebalancing (GPU shape, parallelism) without compromising training accuracy.